Applied Software Testing Project

1 Overview

This assignment is a project-based make-up task for students who missed one or more course activities. It is designed to assess practical understanding of software testing concepts covered in the course.

You will select <u>ONE project</u> and complete ONLY the sections that correspond to the tasks you missed.

The assignment must be completed individually. Students are expected to demonstrate their own understanding of testing concepts, test design, implementation, and reporting.

1.1 Mapping: Activities → Sections

The list below shows how each original course activity corresponds to a section in this project. If you missed any activity, you must complete the matching section(s) instead. This ensures you demonstrate the same knowledge and skills through this project. You only need to complete the sections that correspond to the activities you did not complete.

- **Section 1**→ Exercise 1 (Part 1 & 2), Exercise 3 (Part 1)
- **Section 2**→ Exercise 2 (Part 2)
- **Section 3**→ Exercise 3 (Part 2 – JUnit)
- **Section 4**→ Exercise 4 (Part 1 & 2 – Black-box)
- **Section 5**→ Exercise 5 (Part 1 & 2 – White-box)
- **Section 6**→ Assignment 2 (TDD)

1.2 Quizzes

- **Online Quiz 1 or 2 missed → complete:**
  Section 2 + Section 4 + Section 5

1.3 Rule

Complete only the section(s) that match the activity you missed.

1.4 General Instructions

- Use the selected project consistently in all required sections.

- Sections 3 to 6 must be completed in **Eclipse** by creating a Java project and writing test scripts that test the system.
- The report must include **screenshots** of your work, including code, test execution, and results.
- For all sections, record a **short video presentation** showing your understanding of the work. Upload the video to YouTube as an **unlisted** video and submit the link.
- The report will be checked for **similarity** and **AI-generated text**. Write in your own words and explain your own understanding.
- Reference the **lecture slides** when explaining concepts and designing tests.

---

2 Project Options

Choose **ONE** of the following projects. The same selected project must be used in all sections.

2.1 Project A: Smart Calculator System

A calculator system that supports arithmetic operations on two integers.

System requirements:

- Accept two integer inputs: *a* and *b*
- Support the following operations:
  - Addition
  - Subtraction
  - Multiplication
  - Division
- Division by zero must be handled safely
- Invalid inputs must be rejected or handled with an error message

Example behavior:

- `a = 10`, `b = 2`, operation = division → output `5`
- `a = 10`, `b = 0`, operation = division → output `Error: Division by zero`

Java Code (Eclipse Starter Implementation):

```java
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
```

```java
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public String divide(int a, int b) {
        if (b == 0) {
            return "Error: Division by zero";
        }
        return String.valueOf(a / b);
    }
}
```

2.2 Project B: Student Grading System

A system that assigns grades based on a numeric score.

System requirements:

- Accept one numeric score
- The score must be between 0 and 100
- Output a grade based on the following scale:
  - 90–100 → A
  - 80–89 → B
  - 70–79 → C
  - 60–69 → D
  - 0–59 → F
- Invalid scores must be rejected

Example behavior:

- Input `85` → output `B`
- Input `105` → output `Invalid score`

Java Code (Eclipse Starter Implementation):

```java
public class GradingSystem {

    public String getGrade(int score) {
        if (score < 0 || score > 100) {
            return "Invalid score";
        }
        if (score >= 90) return "A";
```

```
        else if (score >= 80) return "B";
        else if (score >= 70) return "C";
        else if (score >= 60) return "D";
        else return "F";
    }
}
```

2.3 Project C: Online Login System

A system that validates login credentials.

System requirements:

- Accept username and password
- Verify credentials against stored valid values
- Return `Login successful` when credentials are correct
- Return `Invalid credentials` when credentials are incorrect
- Lock the account after 3 failed attempts
- A locked account must remain locked even if correct credentials are later entered

Example behavior:

- Correct username and password → output `Login successful`
- Three failed attempts → output `Account locked`

Java Code (Eclipse Starter Implementation):

```
public class LoginSystem {

    private int attempts = 0;
    private boolean locked = false;

    public String login(String username, String password) {

        if (locked) {
            return "Account locked";
        }

        if (username.equals("admin") && password.equals("1234")) {
            attempts = 0;
            return "Login successful";
        }

        attempts++;
```

```
        if (attempts >= 3) {
            locked = true;
            return "Account locked";
        }

        return "Invalid credentials";
    }
}
```

3 SECTIONS:

SECTION 1 — TESTING ACTIVITIES AND TEST DOCUMENTATION

In this section, you will demonstrate your understanding of software testing activities and basic test documentation.

TASKS

For your selected project: 1. Explain the role of software testing in the software development lifecycle (SDLC). 2. Identify and briefly explain the following testing levels:

- Unit Testing
- Integration Testing
- System Testing

3. Prepare a short **test documentation summary** for your project.
4. Provide a basic **test case table** with at least **5 test cases**.

Required content

Your section must include:

- Purpose of testing
- Scope of testing
- Test environment or assumptions
- Test case ID
- Test objective
- Input data
- Expected output
- Actual output if available
- Pass/Fail result

SECTION 2 — SYSTEM REQUIREMENTS, TEST SCENARIOS, TEST PROCEDURES, AND TEST SUITES

This section must follow a **systematic test design approach** based on the system requirements of the selected project.

Tasks

Using the requirements of your selected project: 1. Identify the main **functional requirements**. 2. Derive **test scenarios** from the requirements. 3. Convert the scenarios into detailed **test procedures**. 4. Group

the procedures into **test suites**. 5. Explain how your test design covers both normal and abnormal behavior.

Required content

Your section must include the following items.

2.1 Requirements Analysis

List the system requirements clearly and number them.

2.2 Test Scenarios

For each requirement, write one or more test scenarios. A test scenario should describe a realistic situation to be tested, such as valid input, invalid input, boundary input, or exception handling.

2.3 Test Procedures

For each scenario, write a step-by-step procedure that explains exactly how the test will be performed.

Each test procedure must include:

- Test procedure ID
- Scenario reference
- Preconditions
- Test steps
- Test data
- Expected result

2.4 Test Suites

Organize the test procedures into logical test suites.

Examples of possible test suites:

- Valid input suite
- Invalid input suite
- Boundary value suite
- Error-handling suite
- Authentication suite
- Arithmetic operation suite

2.5 Test Case Table

Provide a complete test case table with at least **10 test cases**.

Each test case must include:

- Test case ID

- Requirement reference
- Scenario reference
- Test procedure reference
- Input data
- Expected result
- Priority
- Pass/Fail result

Important requirement

This section must show a clear connection between:

- system requirements
- test scenarios
- test procedures
- test suites
- test cases

The goal is to demonstrate systematic test design, not just a list of random test cases.

SECTION 3 — UNIT TESTING IN ECLIPSE

This section must be implemented in **Eclipse**.

Tasks

1. Create a Java project in Eclipse for your selected system.
2. Write the program code for the selected project.
3. Write **JUnit test cases** to test the core functionality.
4. Run the tests in Eclipse and show the results.
5. Include screenshots of:
    - Project structure
    - Source code
    - JUnit test code
    - Test execution output

Requirements

Your test code must include:

- At least **3 to 5 JUnit test methods**
- Meaningful test names
- Tests that cover normal behavior and at least one invalid or edge case

Report content

Explain:

- What the tests verify
- Why these test cases were selected
- How the tests relate to the requirements

SECTION 4 — BLACK-BOX TESTING USING EP AND BVA IN ECLIPSE

This section must be implemented in **Eclipse** and documented in the report.

Tasks

1. Identify relevant **equivalence partitions** for the selected project.
2. Identify relevant **boundary values**.
3. Create test scripts or test methods in Eclipse to test these cases.
4. Run the tests and include the results in the report.

Requirements

Your work must include:

- Clear explanation of equivalence partitioning
- Clear explanation of boundary value analysis
- A table of partitions and boundary values
- Test cases derived from these techniques

Report content

Include screenshots showing:

- The identified partitions
- The boundary values
- The JUnit test cases or test scripts
- The execution results

SECTION 5 — WHITE-BOX TESTING: STATEMENT AND BRANCH COVERAGE IN ECLIPSE

This section must be implemented in **Eclipse** and must demonstrate white-box testing.

Tasks

1. Use a code fragment from the selected project or a small supporting method written for the project.
2. Identify the statements and branches in the code.
3. Design test cases to achieve:
    - Statement coverage
    - Branch coverage
4. Execute the tests in Eclipse and include the results.

Requirements

Your code must contain a visible control structure such as:

- `if`
- `else if`
- `switch`
- loop logic, if appropriate

Report content

Include:

- The code under test
- A short explanation of the control flow
- A statement/branch analysis
- The test cases used for coverage
- Screenshots of the code and test results

Note

The purpose of this section is to show that you can design tests based on the internal structure of the code, not only on external behavior.

SECTION 6 — TEST-DRIVEN DEVELOPMENT (TDD) IN ECLIPSE

This section must be implemented in **Eclipse** and presented through a video demonstration.

Tasks

1. Add a new feature to your selected project using **Test-Driven Development (TDD)**.
2. Follow the TDD cycle strictly:
    - Write a failing test first
    - Run the test (it must fail)
    - Implement the minimum code required to pass the test
    - Run the test again (it must pass)
    - Refactor the code if needed
3. Clearly demonstrate the full process in your video.

Important Video Requirement (Expanded Guidance)

Your video must clearly show **your understanding process**, not just screen recording.

The video should be structured into TWO possible paths depending on which section(s) you are completing:

3.1.1 A) If you are completing Section 3–6 (Coding-Based Sections)

Your video must include:

1. Opening Eclipse project (show project structure clearly)
2. Navigating to the source code and test folders
3. Running the test and clearly showing failure output
4. Running the test again to show it passes
5. Short explanation of:
    - Why the test failed initially
    - How the implementation fixed it
    - How this follows the TDD cycle
6. For section 6 show how to use TDD. Follow the TDD cycle strictly:
    - Write a failing test first
    - Run the test (it must fail)
    - Implement the minimum code required to pass the test
    - Run the test again (it must pass)
    - Refactor the code if needed

3.1.2 B) If you are ONLY completing Section 1 or Section 2 (Non-coding or Design Sections)

You MUST still submit a video, but the focus is different.

Your video must include:

1. Opening your report (PDF or Word document)
2. Explaining your **test design work** clearly:
    - Section 1: Explain your test cases and why they were selected
    - Section 2: Explain requirements → scenarios → procedures → test suites flow
3. Showing at least one example test case or scenario in detail
4. Speaking explanation (not just reading slides)

4 SUBMISSION REQUIREMENTS

Your submission must include the following:

1. **A written report** in PDF format **includes Screenshots** from Eclipse and test execution. The report will be checked for similarity and AI-generated content.
2. **An unlisted YouTube video link** demonstrating your work

VIDEO QUALITY REQUIREMENTS

- Video must be uploaded to YouTube as **unlisted**
- Must include your voice explanation (no silent videos)
- Must clearly show screen actions (Eclipse or report document)
- Duration: short but sufficient (recommended 5–10 minutes)

REPORT CONTENT

Include:

- TDD explanation (for coding students)
- Screenshots of failing test
- Screenshots of implementation
- Screenshots of passing test
- YouTube unlisted link

REPORT FORMAT

The report must be clearly structured and must include these sections:

- First page: Your detail, missing tasks, and YouTube link
- Section 1
- Section 2
- Section 3
- Section 4
- Section 5
- Section 6

5 Assessment Notes

Your work will be assessed based on:

- Correctness of test design
- Quality of test procedures and test suites
- Correct implementation in Eclipse
- Accuracy of JUnit tests
- Understanding of black-box and white-box testing
- Quality of the TDD demonstration
- Clarity of the report
- Use of screenshots and evidence
- Proper reference to lecture slides
- Originality of writing and presentation

The result of this make-up task will be either accepted or rejected. If accepted, you will receive a passing score for the corresponding task(s).

6 Important Academic Integrity Note

Your submission must be your own work. The report and video should reflect your own understanding. **Similarity checking and AI-text detection** may be used to review submissions.